



PicoVNA[®]

Vector Network Analyzers

PicoVNA 5 Programmer's Guide
Python Edition

Contents

1. Introduction	5
1.1. Welcome	5
1.2. Requirements	5
1.2.1. PicoVNA 5 compatible instrument	5
1.2.2. Supported operating systems and platforms	5
1.2.3. Programming language requirements	6
1.2.4. Windows-specific requirements	6
1.2.5. Linux-specific requirements	6
1.3. Obtaining the SDK	6
1.4. Example programs	6
1.5. Software license conditions	7
1.6. Trademarks	7
1.7. Contacting us and obtaining support	7
2. Getting started	9
2.1. Environment setup	9
2.1.1. All platforms: installing the Python package	9
2.1.2. Additional platform-specific libraries: Windows	9
2.1.3. Additional platform-specific libraries: Linux	10
2.1.4. Additional platform-specific libraries: macOS	10
2.2. Getting started	10
2.3. Typical program structure	11
2.3.1. Synchronous mode	11
2.3.2. Asynchronous mode	12
3. API overview	15
3.1. Connecting to the instrument (or multiple instruments)	15
3.1.1. Connecting to any available instrument	15
3.1.2. Connecting to a specific instrument	15

3.1.3. Using a simulated demonstration instrument (for evaluation or debugging)	15
3.1.4. Querying instrument metadata (e.g. serial number)	15
3.2. Loading a user calibration	16
3.2.1. Exporting a user calibration from the PicoVNA 5 software	16
3.2.2. Applying a user calibration at runtime via the API	16
3.3. Setting up and performing a measurement	17
3.3.1. Setting up a measurement with uniform frequency step and constant power level	17
3.3.2. Setting up a complex measurement with non-uniform frequency step or swept power level	17
3.4. Retrieving the data resulting from a measurement	18
3.4.1. Collecting measurements synchronously	18
3.4.2. Collecting measurements asynchronously	18
3.4.3. Aborting an in-progress measurement	18
4. API reference	19
4.1. Class: Device	19
4.1.1. open	19
4.1.2. openAny	19
4.1.3. openDemo	20
4.1.4. getInfo	20
4.1.5. isIdle	20
4.1.6. getTemperature (PicoVNA 108 only)	21
4.1.7. loadFactoryCalibration	21
4.1.8. applyCalibrationFromFile	21
4.1.9. getMetadataForCurrentCalibration	21
4.1.10. pulseTriggerOutput	22
4.1.11. trigger	22
4.1.12. performMeasurement	22
4.1.13. startMeasurement	22
4.2. Class: MeasurementConfiguration	22
4.2.1. MeasurementConfiguration (constructor)	22
4.2.2. getTriggerMode	23
4.2.3. setTriggerMode	23
4.2.4. getTriggerAction	23
4.2.5. setTriggerAction (PicoVNA 108 only)	23
4.2.6. clear	24

4.2.7. addPoint	24
4.2.8. setAveraging	24
4.2.9. setPortOffset	25
4.2.10. setDeEmbedPortNetworks	25
4.2.11. getPoint	25
4.2.12. getPoints	25
4.2.13. getMeasurementFrequencies	26
4.2.14. numPoints	26
4.2.15. addUniformFrequencySweep	26
4.2.16. addUniformPowerSweep	26
4.3. Class: ActiveMeasurement	27
4.3.1. hasMorePoints	27
4.3.2. getConfig	27
4.3.3. getNextPoint	27
4.3.4. tryGetNextPoint	27
4.3.5. getAllPoints	27
4.3.6. abort	28
4.4. Function group: Time Domain	28
4.4.1. class TimeDomainOptions	28
4.4.2. struct TimeDomainSample	28
4.4.3. transform	29
4.5. Function group: Enhancement	29

Chapter 1

Introduction

1.1 Welcome

The PicoVNA Vector Network Analyzers from Pico Technology are compact, high-performance measuring instruments. A VNA works by driving a swept sine wave test signal into one port of a DUT and measuring the reflected (and optionally the transmitted) signals. The signal may then be injected into the other port of the DUT for a further set of measurements. The VNA then uses these measurements to calculate the DUT's S-parameters.

This manual describes the operation of the PicoVNA 5 libraries available to support the PicoVNA 106 and PicoVNA 108 Vector Network Analyzers. The key intention of the libraries is to provide remote automation under standard programming environments such as C++ or Python, or specialist test and measurement, scientific and math environments such as National Instruments LabVIEW and MathWorks MATLAB.

1.2 Requirements

1.2.1 PicoVNA 5 compatible instrument

The PicoVNA 5 Software Development Kit (SDK) requires a PicoVNA instrument that is compatible with the PicoVNA 5 software. To check if your instrument is compatible, download and run the PicoVNA 5 software. You will have the opportunity to upgrade your instrument using the PicoVNA 5 software if it is not currently compatible. If your PicoVNA instrument is not compatible with the PicoVNA 5 software, you can use the SDK in demonstration mode for evaluation purposes.

1.2.2 Supported operating systems and platforms

- Windows 10 and 11 (x64)
- Linux (x64 and aarch64)

The following list of distributions and platforms have been tested, although the PicoVNA 5 SDK is designed to work on any Linux distribution released within the last ten years: Debian 8 ("jessie") and later versions (x64), Ubuntu 18.04 (LTS) and later versions (x64), Linux Mint Cinnamon 21.1 (Vera) and later versions (x64), openSUSE Leap 15.0 and later versions (x64), Fedora 28 and later versions (x64), Arch Linux (x64), Raspbian (Raspberry Pi aarch64)

- macOS (Intel x64 and ARM aarch64)

1.2.3 Programming language requirements

- PicoVNA 5 SDK (downloadable)
Has language bindings for Python
- Python 3.8 or higher

1.2.4 Windows-specific requirements

On Windows, if the PicoVNA 5 software is not installed, the installer `VNA5_SDK.exe` (obtained as part of the downloadable SDK) must be run. This installer installs the kernel drivers required to communicate with the PicoVNA instruments. This step is not required if the PicoVNA 5 software is installed (as the drivers are installed as part of the PicoVNA 5 software installation process). Running an installer for the SDK is not required on Linux or macOS.

1.2.5 Linux-specific requirements

On Linux, a `udev` rule must be installed. If the PicoVNA 5 software is installed, this will be done automatically. Otherwise, create a file called `96-picovna.rules` in your `udev` rules directory (typically `/etc/udev/rules.d`) with the following contents:

```
ATTRS{idVendor}=="0ce9", MODE="0666"  
ATTRS{idVendor}=="0403", ATTRS{idProduct}=="6015", ATTRS{manufacturer}==  
"FTDI", ATTRS{product}=="FT240X USB FIFO", MODE="0777" RUN="/bin/bash -c  
'echo \"${DEVPATH}\" | /bin/sed -E \"s,.*(\\[0-9.-\\+\\])(\\.\\.\\.?)?,\\1:1.0,\" >  
/sys/bus/usb/drivers/ftdi_sio/unbind'
```

You can also copy this file directly from the downloadable SDK.

If you wish to use the PicoVNA instrument immediately after copying this file, you may need to run the following command to reload the `udev` rules:

```
sudo udevadm control --reload-rules && sudo udevadm trigger
```

1.3 Obtaining the SDK

The SDK is distributed as a zip file, which can be downloaded from https://www.aairobotics.com/vna5_internal_builds/temp/vna5_sdk.zip.

The SDK contains all the programming language-specific files that you need to programmatically control your PicoVNA 5 instrument. It does not contain example programs, which can be obtained from GitHub (see Section 1.4).

1.4 Example programs

Example programs using the API can be downloaded from https://github.com/AAI-Robotics/PicoVNA5_SDK_Examples.

1.5 Software license conditions

The material contained in this release is licensed, not sold. Pico Technology Limited grants a license to the person who installs this software, subject to the conditions listed below.

Access. The licensee agrees to allow access to this software only to persons who have been informed of these conditions and agree to abide by them.

Copyright. AAI Robotics Ltd and Pico Technology Ltd. jointly claim the copyright of, and retains the rights to, all SDK materials (software, documents, etc.). You may copy and distribute SDK files without restriction, as long as you do not remove any copyright statements. The example programs may be modified, copied and distributed for the purpose of developing programs to collect data using Pico products.

Liability. AAI Robotics Ltd and Pico Technology Ltd and their agents shall not be liable for any loss, damage or injury, howsoever caused, related to the use of Pico Technology equipment or software, unless excluded by statute.

Fitness for purpose. As no two applications are the same, Pico Technology cannot guarantee that its equipment or software is suitable for a given application. It is your responsibility, therefore, to ensure that the product is suitable for your application.

Mission-critical applications. This software is intended for use on a computer that may be running other software products. For this reason, one of the conditions of the license is that it excludes use in mission-critical applications, for example life support systems.

Viruses. This software was continuously monitored for viruses during production, but you are responsible for virus-checking the software once it is installed.

Support. If you are dissatisfied with the performance of this software, please contact our technical support staff, who will try to fix the problem within a reasonable time. If you are still dissatisfied, please return the product and software to your supplier within 14 days of purchase for a full refund.

Upgrades. We provide upgrades from our website at www.picotech.com. We reserve the right to charge for updates or replacements.

1.6 Trademarks

Pico Technology is an internationally registered trademark of Pico Technology Ltd. PicoVNA and PicoSDK are registered trademarks of Pico Technology Ltd.

Windows, Excel and Visual Basic for Applications are registered trademarks or trademarks of Microsoft Corporation in the USA and other countries. LabVIEW is a registered trademark of National Instruments Corporation. MATLAB is a registered trademark of The MathWorks, Inc.

1.7 Contacting us and obtaining support

Pico provides tailored technical and sales support to all customers. Contact us via our website, www.picotech.com, or using the details below.

Technical support forum: <https://picotech.com/support/forum141.html>

**United Kingdom
(global headquarters)**

Pico Technology
James House
Colmworth Business Park
St Neots
Cambridgeshire
PE19 8YP
United Kingdom

+44 (0) 1480 396 395
support@picotech.com
sales@picotech.com

UK office hours:

09:00 to 17:00 (UK time)
Monday to Friday

North America regional office

Pico Technology
320 N Glenwood Blvd
Tyler
TX 75702
United States

+1 800 591 2796
support@picotech.com
sales@picotech.com

North America office hours:

09:00 to 17:00 (Central time)
Monday to Friday

Asia-Pacific regional office

Pico Technology
Room 2252, 22/F, Centro
568 Hengfeng Road
Zhabei District
Shanghai 200070
PR China

+86 21 2226-5152
pico.asia-pacific@picotech.com

Chapter 2

Getting started

2.1 Environment setup

To write your own Python programs using the PicoVNA 5 Application Programming Interface (API), or to run the example programs in the PicoVNA 5 SDK, it is necessaryL

1. Install the `vna` Python package (obtained from PyPI)
2. Obtain some additional platform-specific libraries: from the downloadable SDK, and ensure they are installed in locations that the Python interpreter can find them.

2.1.1 All platforms: installing the Python package

The `vna` Python package should be installed in the usual way via PyPI. Use:

```
pip install vna
```

2.1.2 Additional platform-specific libraries: Windows

On Windows, you must take the correct files from the PicoVNA 5 SDK for your version of Python (Python 3.8, 3.9, 3.10 and 3.11 are supported). Make sure to copy the correct libraries from the SDK for your version of Python.

For Python version 3.X, copy files from `vna5_sdk/python/windows/python3X` (where X may be 8, 9, 10 or 11).

If you do not know what version of Python you are using, the following Python code will tell you:

```
import sys
print(f"{sys.version}")
```

Alternatively, type `python3 --version` at the command prompt.

The files that must be taken from the SDK are:

- `_vna_python.pyd`

- `vna.lib`
- `vna_python.lib`
- `vna.dll`
- `ftd2xx.dll`

For simple projects, place all these files in the same directory as your own Python script.

2.1.3 Additional platform-specific libraries: Linux

The files that must be taken from the SDK are:

- `_vna_python.so`
- `libvna.so`
- `libftd2xx.so`

For simple projects, place all these files in the same directory as your own Python script.

2.1.4 Additional platform-specific libraries: macOS

The files that must be taken from the SDK are:

- `_vna_python.dylib`
- `libvna.dylib`
- `libftd2xx.dylib`

For simple projects, place all these files in the same directory as your own Python script.

2.2 Getting started

The best way to get started is by running the example programs. See Section 1.4 for details on how to obtain them.

If you do not have the PicoVNA 5 software installed, ensure that you have installed any necessary drivers and/or udev rules (see Sections 1.2.4 and 1.2.5).

In this section, we will run the example program `01_simple_frequency_sweep.py`. The procedure for running the other example programs is analogous.

Assuming the procedure in 2.1 has been followed to set up the environment, the example program can be run using:

```
cd /path/to/01_simple_frequency_sweep
python3 01_simple_frequency_sweep.py
```

If you see the error `ModuleNotFoundError: No module named vna`, then the `vna` package has not been correctly installed from PyPI.

If you see an error similar to `ImportError: ...: No such file or directory` then `_vna_python.so` (or `_vna_python.pyd` on Windows, or `_vna_python.dylib` on macOS) is not being found. On Windows, make sure `_vna_python.pyd` is in a suitable location. On Linux, the easiest workaround is to set the `LD_LIBRARY_PATH` environment variable to the directory it is found in:

```
LD_LIBRARY_PATH=example python example.py
```

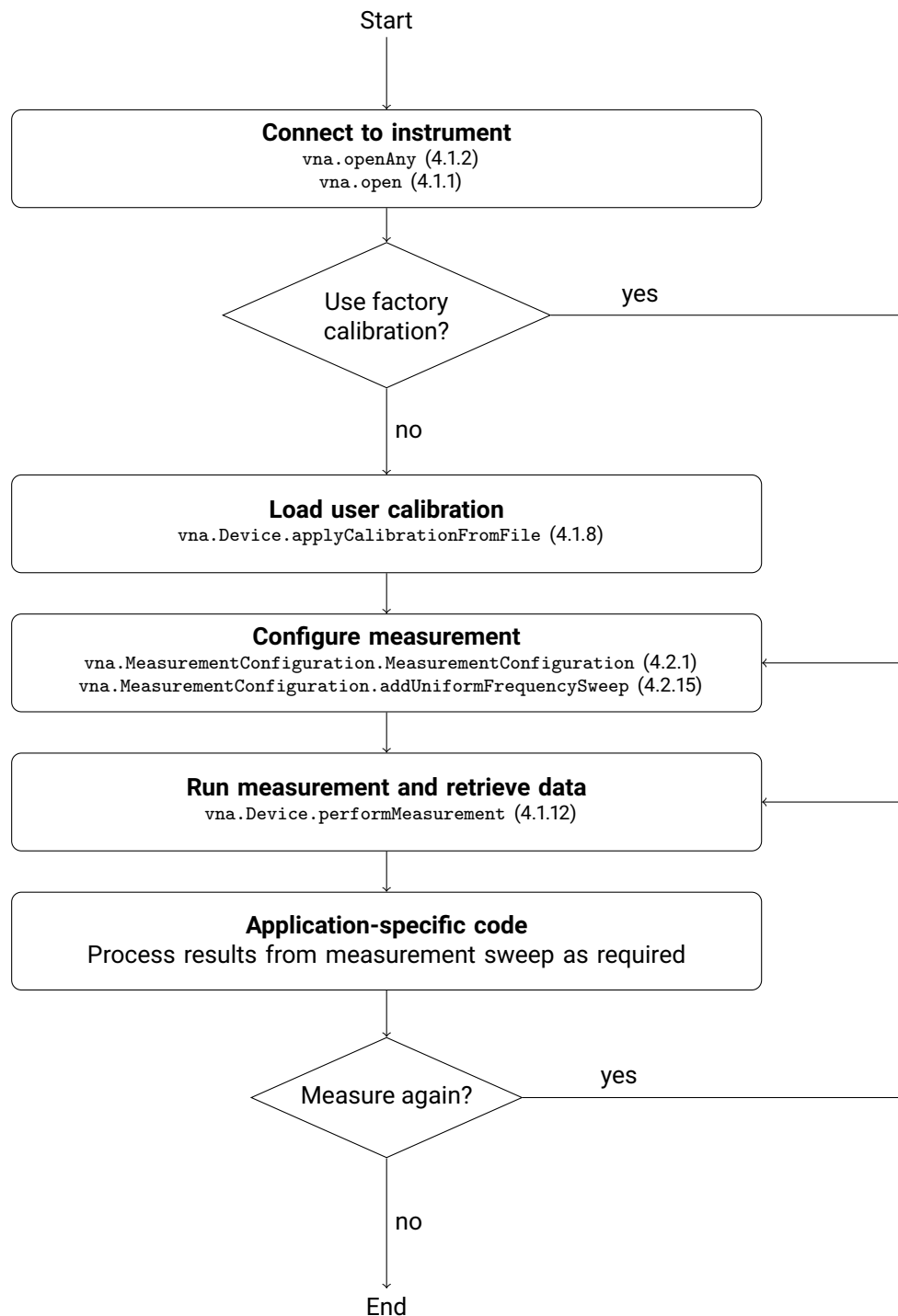
The equivalent command on macOS is:

```
DYLD_LIBRARY_PATH=example python example.py
```

2.3 Typical program structure

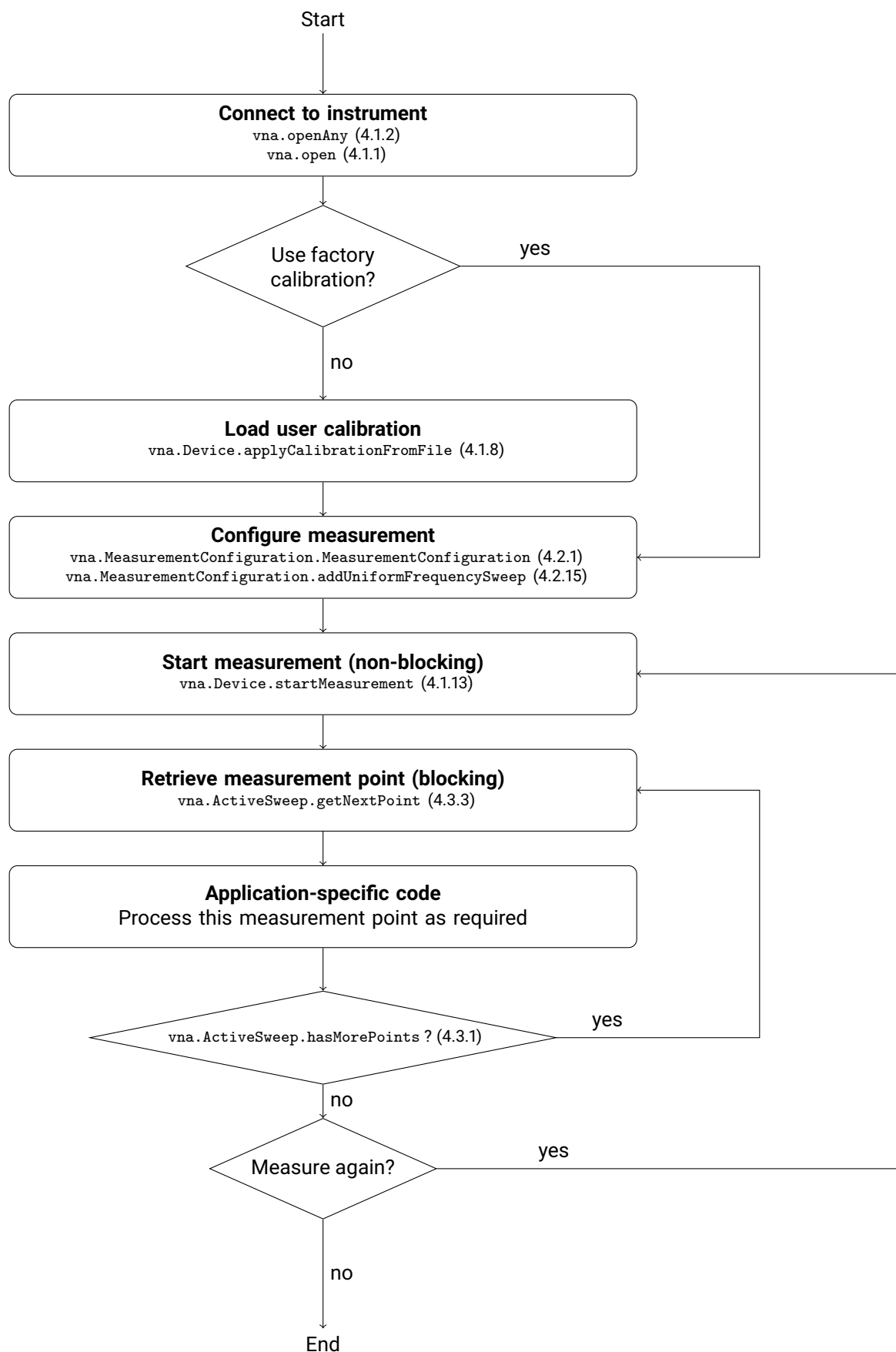
2.3.1 Synchronous mode

The diagram below shows the steps to carry out a measurement using synchronous mode (i.e. waiting for the entire measurement sweep to complete before processing measurements). It is simplified and incorporates the minimum number of steps to perform a measurement.



2.3.2 Asynchronous mode

The diagram below shows the steps to carry out a measurement using asynchronous mode (i.e. processing measurements point-by-point, rather than waiting for the whole measurement sweep to complete). It is simplified and incorporates the minimum number of steps to perform a measurement.



Chapter 3

API overview

3.1 Connecting to the instrument (or multiple instruments)

3.1.1 Connecting to any available instrument

If a only a single PicoVNA instrument is available, the simplest way to connect to it is to use the `vna.Device.openAny` function (4.1.2). The function returns a `vna.Device` object that can then be used for instrument configuration and performing measurements.

This function may also be used if many instruments are available and required, since the device metadata can be queried after connecting, and the instruments can be distinguished by their serial numbers.

3.1.2 Connecting to a specific instrument

To connect to an instrument with a specific serial number, use the `vna.Device.open` function (4.1.1). The serial number of the instrument to open may be specified as a parameter.

3.1.3 Using a simulated demonstration instrument (for evaluation or debugging)

To open a simulated demonstration instrument for evaluation or debugging, use the `vna.Device.openDemo` function (4.1.3). The simulated DUT to use can be specified as a parameter; available options are a bandpass filter, a lowpass filter, an attenuator, or an antenna (S11 only).

Please note the calibration functions will not be available when using the simulated demonstration device.

3.1.4 Querying instrument metadata (e.g. serial number)

The intrinsic properties of a connected instrument can be inspected using the `Device.getInfo` function (4.1.4). This returns a `vna.DeviceInfo` object that includes metadata including the device serial, model and information about its capabilities.

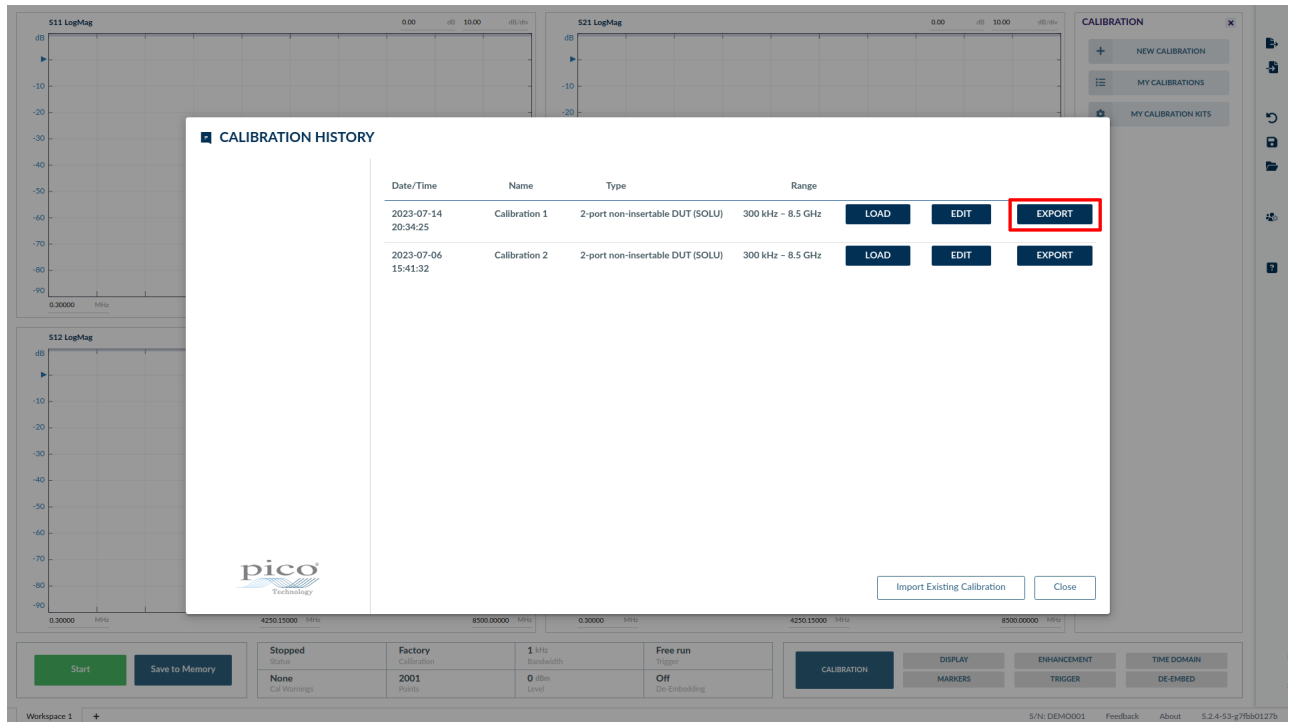


Figure 3.1: Exporting a calibration from the PicoVNA 5 software.

3.2 Loading a user calibration

3.2.1 Exporting a user calibration from the PicoVNA 5 software

It is not currently possible to perform a calibration via the API. Instead, a calibration must be:

- (*in advance*) Performed within the PicoVNA 5 software
- (*in advance*) Exported to a file from the PicoVNA 5 software
- (*at user program runtime, every run*) Applied via the API

If no user calibration is loaded at runtime, the factory calibration will be used.

To export a user calibration from the PicoVNA 5 software, go to Calibrations → My Calibrations and press the “Export” button associated with the relevant calibration. You will then be given the opportunity to save your calibration to a file, which can be applied via the API. Figure 3.1 illustrates how the user calibration named Calibration 1 would be exported.

3.2.2 Applying a user calibration at runtime via the API

Use the `Device.applyCalibrationFromFile` function (4.1.8) to load the user calibration file that was exported following the instructions in Section 3.2.1. This function can be passed an absolute or relative path to the calibration to be applied.

The user calibration to be loaded must have been performed from the PicoVNA 5 software (i.e. it must be a `.calx` file). It is not possible to load calibrations that were performed by the PicoVNA 3 software (from `.cal` files) via the API.

3.3 Setting up and performing a measurement

The parameters for a measurement must be specified before the measurement is carried out. Even when a user calibration has been applied, the parameters for the measurement must be set explicitly. If the measurement parameters set do not match those of the applied calibration, the calibration will be automatically interpolated without warning.

A measurement is configured using the `MeasurementConfiguration` object (4.2). To initiate the measurement once it has been configured, pass the `MeasurementConfiguration` object to the `Device.performMeasurement` function (4.1.12).

3.3.1 Setting up a measurement with uniform frequency step and constant power level

A convenient helper function `MeasurementConfiguration.addUniformFrequencySweep` (4.2.15) is available to set up a measurement with uniform frequency step and constant power level.

For example, to set up a 1001-point measurement, from 1 GHz to 2 GHz, with a power level of 0.0 dBm and 1000 Hz bandwidth:

```
mc = vna.MeasurementConfiguration()
mc.addUniformFrequencySweep(
    1001,
    1e9,
    2e9,
    0.0,
    1000
)
```

3.3.2 Setting up a complex measurement with non-uniform frequency step or swept power level

A measurement can be set up point-by-point, so that each measurement point can use a different frequency and/or power level. All points in the sweep must be measured using the same bandwidth.

For example, to set up a 3-point measurement, measuring a frequency of 1 GHz at three different power levels:

```
mc = vna.MeasurementConfiguration()

pt = vna.MeasurementPoint()
pt.frequencyHz = 1e9
pt.powerLeveldBm = -20.0
pt.bandwidthHz = 1000
mc.addPoint(pt)

pt.powerLeveldBm = 0.0
pt.addPoint(pt)

pt.powerLeveldBm = 6.0
pt.addPoint(pt)
```

3.4 Retrieving the data resulting from a measurement

The `Device.performMeasurement` function returns a `ActiveMeasurement` object (4.3) that can be queried to:

- Check the status of the measurement. Has it completed yet?
- Retrieve the resulting data from the measurement.

Two modes are available for retrieving data from the `ActiveMeasurement`: synchronous mode and asynchronous mode. In synchronous mode, the function used to retrieve data from the device will block until all the data from all measurement points are available. In asynchronous mode, measurements can be retrieved point-by-point, and processed in parallel with the device collecting more measurement points. In asynchronous mode, the function used to retrieve data from the device will only block until the next measurement point is available.

3.4.1 Collecting measurements synchronously

Use the `ActiveMeasurement.getAllPoints` function (4.3.5) to retrieve all the measurements in one `list`. The function will block until all the measurements are available.

3.4.2 Collecting measurements asynchronously

To retrieve the data from measurement points one-by-one, so that data can be processed as soon as it is available without waiting for the sweep to complete, use the `ActiveMeasurement.getNextPoint` function. This function will block until the data from the next point in the measurement sweep is available.

A non-blocking function, `ActiveMeasurement.tryGetNextPoint` (4.3.4), is also available. This will return data if and only if it is not available without blocking the thread.

3.4.3 Aborting an in-progress measurement

Use the `ActiveMeasurement.abort` function (4.3.6) to cancel a measurement that is currently in progress.

Chapter 4

API reference

4.1 Class: Device

Description:

The `Device` represents a connection to a VNA attached to this computer.

4.1.1 open

Declaration:

```
@staticmethod  
def open(serial)
```

Description:

Open the instrument with the given serial number.

Opening the same device twice is not allowed. An instrument may be closed by destructing the object the was created when it was opened (there is no explicit function for closing an instrument).

For testing and product evaluation, serial numbers of the form `DEMOXXXX` (where `XXXX` is some integer) may be used to request a simulated demonstration device. This provides a convenient way to test your code where no real device is available.

Parameters:

`serial` The serial of the instrument to open.

Returns:

An object that can be used to control the instrument that has been opened.

4.1.2 openAny

Declaration:

```
@staticmethod  
def openAny()
```

Description:

Open any real instrument, but not the demo device.

This is a convenient function to open the instrument where only one instrument is expected to be connected to the computer, and your program only requires one instrument.

Since opening the same device twice is not allowed, it is not possible to use this function repeatedly to open multiple devices. To achieve that, you must get a list of available devices, and then open them by serial number.

Parameters:

This function takes no parameters.

Returns:

An object that can be used to control the instrument that has been opened.

4.1.3 openDemo

Declaration:

```
@staticmethod
def openDemo(*args)
```

Description:

Open the simulated demo device.

This is equivalent to calling `open("DEM0000")`, except this function also allows the simulated DUT to be selected from a set of options.

Parameters:

`dut` `int` (optional) The device under test to demo.

The following constants are defined that can be passed to the `dut` parameter:

- `DemoDeviceDut_LOW_PASS_FILTER`
Demo device will replay data recorded from measurements of a low pass filter
- `DemoDeviceDut_BAND_PASS_FILTER`
Demo device will replay data recorded from measurements of a band pass filter
- `DemoDeviceDut_ATTENUATOR`
Demo device will replay data recorded from measurements of an attenuator
- `DemoDeviceDut_ANTENNA`
Demo device will replay data recorded from measurements of an antenna (S11 only)

Returns:

An object that can be used to control the simulated instrument that has been opened.

4.1.4 getInfo

Declaration:

```
def getInfo(self)
```

Description:

Query properties of the connected instrument.

The returned struct describes immutable, intrinsic properties of the attached instrument, such as model number and various hardware limits.

Parameters:

This function takes no parameters.

4.1.5 isIdle

Declaration:

```
def isIdle(self)
```

Description:

Returns true iff the device is not currently collecting measurements.

Parameters:

This function takes no parameters.

4.1.6 getTemperature (PicoVNA 108 only)

Declaration:

```
getTemperature(self)
```

Description:

Returns the instrument's internal temperature in degrees centigrade.

This function is only available on the PicoVNA 108 instrument; on other instruments a `OperationNotSupportedException` will be raised. If the instrument is still initialising, NaN will be returned. In this case, call this function again later to obtain a temperature reading.

Parameters:

This function takes no parameters.

4.1.7 loadFactoryCalibration

Declaration:

```
def loadFactoryCalibration(self)
```

Description:

Apply the factory calibration.

Parameters:

This function takes no parameters.

4.1.8 applyCalibrationFromFile

Declaration:

```
def applyCalibrationFromFile(self, path)
```

Description:

Apply the calibration stored in the given file path.

The user calibration to be loaded must have been performed from the PicoVNA 5 software (i.e. it must be a `.calx` file). It is not possible to load calibrations that were performed by the PicoVNA 3 software (from `.cal` files) via the API.

To export a calibration from the PicoVNA 5 software to a file, so this command can be used to load it, use `Export` feature in the Calibration History modal, accessed via the `Calibrations -> My Calibrations` menu.

Parameters:

`path` Absolute or relative path to the `.calx` file storing the calibration to apply.

4.1.9 getMetadataForCurrentCalibration

Declaration:

```
def getMetadataForCurrentCalibration(self)
```

Description:

Returns metadata for the calibration that is currently applied.

Parameters:

This function takes no parameters.

4.1.10 pulseTriggerOutput

Declaration:

```
def pulseTriggerOutput(self)
```

Description:

Send a pulse through the VNA's trigger output port.

Parameters:

This function takes no parameters.

4.1.11 trigger

Declaration:

```
def trigger(self)
```

Description:

If any sweep plans have been started with `startTriggeredSweep` in `MANUAL` mode, this function will activate them. Otherwise, this function has no effect.

4.1.12 performMeasurement

Declaration:

```
def performMeasurement(self, sweep)
```

Description:

Collect all the measurement points described by the given `vna::MeasurementConfiguration`, and return them in an `std::vector`.

This function blocks until the device has collected all of the measurements, which could take any time from several microseconds to several minutes depending on the `vna::MeasurementConfiguration`.

If you wish to consume the data points incrementally as they become available (rather than blocking your application), use `startMeasurement()` to perform an asynchronous sweep.

4.1.13 startMeasurement

Declaration:

```
def startMeasurement(self, sweep)
```

Description:

Asynchronously start the sweep described by the given sweep plan. The sweep is handed to the sweep scheduler, and its data will become available soon. Use the functions of the `ActiveMeasurement` class to consume the data points incrementally as they become available.

Returns:

An `ActiveMeasurement` object representing the started sweep. The data for this sweep may be extracted from this object.

4.2 Class: MeasurementConfiguration

Description:

Describes a sweep to be performed by the device.

4.2.1 MeasurementConfiguration (constructor)

Declaration:

```
def __init__(self, *args)
```

Description:

Create an empty MeasurementConfiguration.

This is useful if you plan to use `addPoint()` to add a custom set of measurement points to the sweep (as opposed to using `MeasurementConfiguration::frequency()` and friends to generate a uniform sweep).

4.2.2 getTriggerMode

Declaration:

```
def getTriggerMode(self)
```

Description:

When configuring a triggered sweep, what trigger event will cause the sweep to start?

Parameters:

This function takes no parameters.

4.2.3 setTriggerMode

Declaration:

```
def setTriggerMode(self, triggerMode)
```

Description:

For a triggered sweep, configure the trigger event will cause the sweep to start.

Parameters:

`triggerMode` The trigger event that will cause the sweep to start (if a triggered sweep is enabled).

The following constants are defined that can be passed to the `triggerMode` parameter:

- `TriggerMode_FREE_RUN`
The instrument does not wait for a trigger event before beginning each measurement.
- `TriggerMode_RISING_EDGE`
The instrument waits for a rising edge trigger event before beginning each measurement.
- `TriggerMode_FALLING_EDGE`
The instrument waits for a falling edge trigger event before beginning each measurement.
- `TriggerMode_MANUAL`
The instrument waits for the `trigger()` function to be called before beginning each measurement (asynchronous mode only).

4.2.4 getTriggerAction

Declaration:

```
def getTriggerAction(self)
```

Description:

In a triggered sweep, what action will be taken when a trigger event occurs?

Parameters:

This function takes no parameters.

4.2.5 setTriggerAction (PicoVNA 108 only)

CAUTION! This command is experimental. Using it may yield unexpected or incorrect results. The command may be changed or removed in future versions of the API.

Declaration:

```
def setTriggerAction(self, triggerAction)
```

Description:

In a triggered sweep, configure the action will be taken when a trigger event occurs.

Parameters:

`triggerAction` The action that will be taken when a trigger even occurs (if a triggered sweep is enabled).

The following constants are defined that can be passed to the `triggerAction` parameter:

- `TriggerAction_RUN`
The instrument performs the entire measurement in response to a trigger event.
- `TriggerAction_NEXT_POINT`
The instrument measures the next point in the sweep in response to a trigger event.

4.2.6 clear**Declaration:**

```
def clear(self)
```

Description:

Delete all measurement points.

Parameters:

This function takes no parameters.

4.2.7 addPoint**Declaration:**

```
def addPoint(self, p)
```

Description:

Add a point to the sweep. A sweep may be composed of an arbitrary set of points.

It is not necessary for the frequency of each measurement point to exactly match the frequency of error terms in the loaded calibration. Adding points that do not exactly match the frequency of error terms in the loaded calibration will cause a new set of (interpolated) error terms to be generated so that a new calibration need not be carried out. Note that in order to obtain the best instrument capability a new calibration should be performed whenever the sweep parameters change.

Point types (i.e: `MeasurementPoint` or `CWMeasurementPoint`) may not be mixed. At present, all point point types must have the same bandwidth.

Parameters:

`p` The measurement point to add to the sweep.

4.2.8 setAveraging**Declaration:**

```
def setAveraging(self, n)
```

Description:

If $n > 1$, the device will be instructed to perform several sweeps and each measurement point returned by `performMeasurement` or by querying the `ActiveMeasurement` will be the average of the several measurements performed by the device. This averaging process will yield identical results to requesting several sweeps from the device (with averaging switched off) and computing the mean value for each measurement point in your code.

Set $n=1$ to turn off averaging.

Parameters:

- n The number of samples to average over. Must not be zero.

4.2.9 setPortOffset**Declaration:**

```
def setPortOffset(self, port1Length, port2Length, dielectricConstant=1.0)
```

Description:

Apply reference plane offsetting in place.

This is useful to correct for the connection of a device under test via a pair of cables of a known length. This happens before other de-embedding.

Parameters:

- `port1Length` The length, in m, of the cable between port 1 and the device under test. Must be at least 0.
- `port2Length` The length, in m, of the cable between port 2 and the device under test. Must be at least 0.
- `dielectricConstant` The dielectric constant of the medium. This must be at least 1.0.

4.2.10 setDeEmbedPortNetworks**Declaration:**

```
def setDeEmbedPortNetworks(self, port, networks)
```

Description:

Set networks to de-embed from the measurement.

Parameters:

- `port` Port to put the network on.
- `networks` S2P files of networks on the port, starting with the network closest to the port and ending with the network closest to the DUT.

4.2.11 getPoint**Declaration:**

```
def getPoint(self, *args)
```

Description:

Get the i th point from the sweep.

Parameters:

- i Index of the point to get from the sweep (starting from 0).

4.2.12 getPoints**Declaration:**

```
def getPoints(self, *args)
```

Description:

Get the list of all points.

Parameters:

This function takes no parameters.

4.2.13 getMeasurementFrequencies**Declaration:**

```
def getMeasurementFrequencies(self)
```

Description:

Extract the measurement frequencies from all points and return as a list of doubles. This is a handy shortcut for the case where you want to key something by frequency.

Parameters:

This function takes no parameters.

4.2.14 numPoints**Declaration:**

```
def numPoints(self):
```

Description:

Returns the number of points in the configuration.

Parameters:

This function takes no parameters.

4.2.15 addUniformFrequencySweep**Declaration:**

```
def addUniformFrequencySweep(self, numPoints, startFreqHz, stopFreqHz, powerLeveldBm, bandwidthHz)
```

Description:

Convenience function for generating sweeps in which all but the frequency parameter is fixed, and the frequency parameter is moved through a range with equal intervals.

Parameters:

numPoints	Number of points in the sweep.
startFreqHz	The first frequency to measure.
stopFreqHz	The last frequency to measure.
powerLeveldBm	The power level to use.
bandwidthHz	The bandwidth to use.

4.2.16 addUniformPowerSweep**Declaration:**

```
def addUniformPowerSweep(self, numPoints, startPowerLeveldBm, stopPowerLeveldBm, frequencyHz, bandwidthHz)
```

Description:

Convenience function for generating sweeps in which all but the power parameter is fixed, and the power parameter is moved through a range with equal intervals.

Parameters:

numPoints	Number of points in the sweep.
startPowerLeveldBm	The first power to measure.
stopPowerLeveldBm	The last power to measure.
frequencyHz	The frequency to use.
bandwidthHz	The bandwidth to use.

4.3 Class: ActiveMeasurement

Description:

Represents the ongoing execution of a single `MeasurementConfiguration` on the device.

4.3.1 hasMorePoints

Declaration:

```
def hasMorePoints(self)
```

Description:

Determine whether there are more points available to read from the object using `getNextPoint()`.

Not to be confused with `isFinished()`, which merely determines whether the measurement process has finished. If it has, the device is ready to do something else (and you may access all the data from the completed measurement).

Parameters:

This function takes no parameters.

4.3.2 getConfig

Declaration:

```
def getConfig(self)
```

Description:

Get the `MeasurementConfiguration` object originally used to define this sweep.

Parameters:

This function takes no parameters.

4.3.3 getNextPoint

Declaration:

```
def getNextPoint(self)
```

Description:

Read the next data point from this sweep. Blocks until the data is available. This allows you to digest the data as it becomes available, or to manage your own memory.

Parameters:

This function takes no parameters.

4.3.4 tryGetNextPoint

Declaration:

```
def tryGetNextPoint(self, output)
```

Description:

If the next measurement point is already available, write it to `output`.

Parameters:

`output` The measurement point is written here if it is available.

Returns:

True if and only if a point was written out.

4.3.5 getAllPoints

Declaration:

```
def getAllPoints(self)
```

Description:

Wait for the sweep to finish, and return all remaining data at once. Equivalent to calling `getNextPoint()` in a loop.

Parameters:

This function takes no parameters.

4.3.6 abort**Declaration:**

```
def abort(self)
```

Description:

Stop the sweep early.

After stopping the sweep, you can still use `tryGetNextPoint()` to extract any remaining data from its buffer. This function blocks until the sweep has stopped and the device is ready to do other things.

Parameters:

This function takes no parameters.

4.4 Function group: Time Domain

Description:

Perform time domain transforms.

4.4.1 class TimeDomainOptions**Declaration:**

```
class TimeDomainOptions(object):
    def __init__(self):
        ...

    outputStartSeconds = ...
    outputEndSeconds = ...
    minOutputSampleRate = ...
    outputPaddingSamplesStart = ...
    outputPaddingSamplesEnd = ...
    response = ...
    mode = ...
    window = ...
    dcTermination = ...
    dcTerminationResistanceOhms = ...
    kbWindowOrder = ...
```

Description:

Used for specifying configuration options when performing time domain transforms.

4.4.2 struct TimeDomainSample**Declaration:**

```
class TimeDomainSample(object):
    time = ...
    sample = ...
```

4.4.3 transform

Declaration:

```
def transform(*args)
```

Description:

Perform the time domain transform on the given list of points, assuming these points form the entire sweep.

4.5 Function group: Enhancement

4.5.0.1 applySmoothing (1)

Declaration:

```
applySmoothing(*args)
```

Description:

Apply a smoothing algorithm to an array of complex numbers. This is intended to smooth S-parameters.

Parameters:

<code>dst</code>	List of complex numbers. The output.
<code>src</code>	List of complex numbers. The input.
<code>count</code>	The size of the input and output.
<code>amount</code>	The amount of smoothing. This is a number between 0.0 (least smoothing) and 1.0 (most smoothing). Note that this is different from the UI, which ranges from 0.0 to 10.0.

4.5.0.2 applySmoothing (2)

Declaration:

```
applySmoothing(*args)
```

Description:

Apply smoothing in-place.

Parameters:

<code>buffer</code>	The input, which is overwritten with the result.
<code>amount</code>	The amount of smoothing. This is a number between 0.0 (least smoothing) and 1.0 (most smoothing). Note that this is different from the UI, which ranges from 0.0 to 10.0.

UK headquarters:

Pico Technology
James House
Colmworth Business Park
St. Neots
Cambridgeshire
PE19 8YP
United Kingdom

Tel: +44 (0) 1480 396 395

sales@picotech.com
support@picotech.com

North America office:

Pico Technology
320 N Glenwood Blvd
Tyler
TX 75702
United States

Tel: +1 800 591 2796

sales@picotech.com
support@picotech.com

Asia-Pacific office:

Pico Technology
Room 2252, 22/F, Centro
568 Hengfeng Road
Zhabei District
Shanghai 200070
PR China

Tel: +86 21 2226-5152

pico.asia-pacific@picotech.com

EU Authorized Representative:

EU Authorized Representative
Pico Technology GmbH
Im Rehwinkel 6
30827 Garbsen
Germany

Tel: +49 (0) 5131 907 62 90

info.de@picotech.com

www.picotech.com

Copyright © 2017–2023 Pico Technology Ltd

